



CDE2605 UROP

AY24/25 Special Term & AY25/26 Semester 1

Formulation of an edge-coverage algorithm and system for a collaborative multi-robot setup through the Open-RMF framework

Student Name	Matriculation Number
Mayukh Ghosh	A0312084N

Contents

Introduction	1
Methodology	1
System Architecture.....	1
Graph Generation Models.....	2
Algorithm Development	3
Evaluation	3
Simulations and Results	3
Limitations	5
Conclusion.....	5
References	6
Appendices	7
Appendix A: Graph Generation Details	7
Appendix B: Algorithm Pseudocode and Implementation Details	10
Appendix C: Detailed Analysis of CoV Distribution Across Map Instances	14
Appendix D: Gantt Chart.....	16

Introduction

This project aims to improve the efficiency and coordination of multi-robot navigation systems in structured indoor environments. As autonomous mobile robots become more widely used in areas such as logistics, healthcare, and infrastructure maintenance, there is a growing need for frameworks that can manage multiple robots operating in shared spaces. This work builds on the Open Robotic Middleware Framework (Open-RMF), which enables centralized coordination through task dispatching, traffic management, and conflict resolution [1][2]. However, it remains inherently goal-agnostic, creating a key limitation in coverage tasks where deciding where robots should go is as important as coordinating how they move.

The project focuses on the edge coverage problem, where robots must collectively traverse all edges of a graph-represented environment while minimizing redundancy and completion time. Traditional heuristic methods such as frontier-based exploration provide simple boundary-driven goal selection but often suffer from redundancy and inefficient multi-robot coordination. More advanced approaches, including graph partitioning techniques, DRL, GNN-based methods, and algorithms such as DARP, improve efficiency and workload distribution but are generally not designed for real-time integration with middleware systems like Open-RMF, or rely on static task definitions, offline training, or fixed robot assumptions [3][4][5].

To address this gap, this project proposes a graph-aware, adaptive goal-selection algorithm integrated directly into the Open-RMF dispatch pipeline. The method leverages graph structure, robot states, and coverage history to dynamically generate goals that reduce redundant traversal while maintaining low computational overhead and balanced task allocation. The system was validated through simulations and physical experiments using TurtleBot3 platforms, demonstrating significant improvements in coverage efficiency and scalability over baseline methods.

Methodology

System Architecture

This work proposes a system architecture for coordinated multi-robot operation within Open-RMF, integrating goal selection, navigation, and system monitoring. Custom scripts generate goals on maps that are either manually designed or randomly created, with all environments converted into NetworkX graphs for processing [6]. Open-RMF handles route planning and conflict resolution, while Navigation2 executes robot motion.

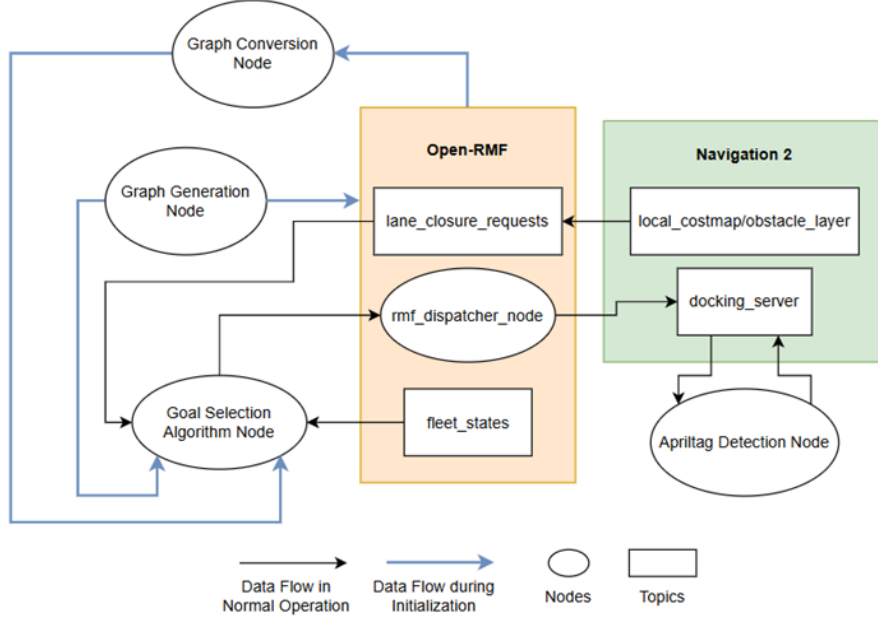


Figure 1: Proposed System Architecture for edge-coverage applications

To support dynamic environments, robots publish detected obstacles to a lane-closure topic, allowing both Open-RMF and the goal-selection algorithm to avoid blocked paths as shown in Figure 1. Battery management is automated through charge requests, with docking performed using AprilTag-based localization. The system was validated on TurtleBot3 platforms and demonstrated reliable performance, though it remains sensitive to navigation parameter tuning and network stability. Optimal results were achieved using a route server, regulated pure pursuit controller, and behavior trees capable of handling temporary obstacles without prematurely terminating navigation.

Graph Generation Models

Realistic graph-based environments were required to evaluate the algorithm, with key requirements including planarity, connectivity, and structural similarity to real-world layouts such as offices or warehouses. Standard random graph models, including Erdős–Rényi and Barabási–Albert, were unsuitable due to non-planarity and unrealistic spatial structure [7], while grid graphs, though planar, lacked variability [8]. Tree-based models were therefore explored, with lobster graphs selected due to their hierarchical structure, natural branching, and guaranteed planarity [9]. These graphs were further refined by controlling parameters such as spine length and branch depth, and by introducing loops through probabilistic edge additions to increase complexity. Spatial constraints were also applied to prevent node overlap and ensure accurate robot localization. For fair benchmarking, graphs were normalized using average edge weight to reflect traversal cost. Nodes and edges were configured to align with Open-RMF requirements, including bidirectional edges to avoid deadlocks, and leaf-node placement for parking, charging, and robot initialization. All graphs were automatically translated into Open-RMF-compatible maps. Further implementation details regarding graph construction, spatial constraints, and RMF map conversion are provided in Appendix A.

Algorithm Development

The proposed algorithm was designed to balance computational efficiency, coverage completeness, and real-time applicability. It consists of a goal-selection mechanism and a planner loop that coordinates robot behavior. Key graph structures influencing exploration were identified as stray edges, cycles, and leaf nodes as shown in Figure 2. Stray edges are explicitly prioritized to prevent incomplete coverage, while cycles are handled by directing robots to traverse them fully before exiting, reducing redundant looping. Leaf nodes are treated as terminal goals to ensure complete traversal of linear paths.

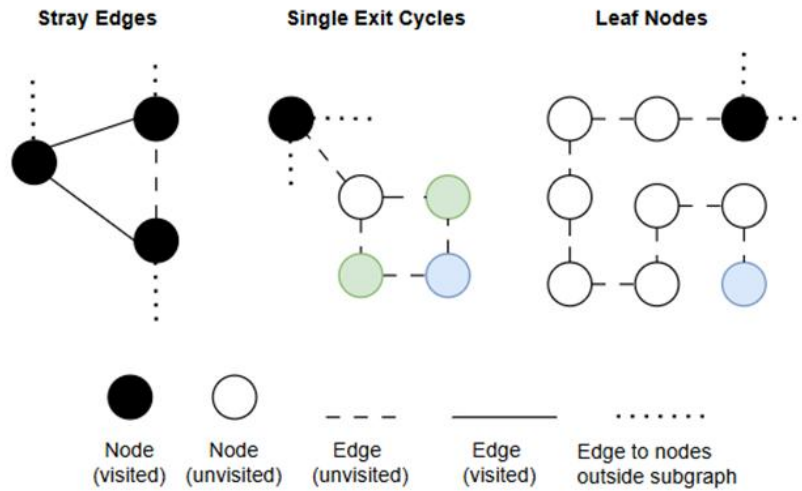


Figure 2: Types of Local Topologies considered

A unified scoring function evaluates candidate goal nodes by balancing exploration gain (unvisited edges) against traversal cost (shortest path distance), with a fragment bonus to prioritize partially explored regions and a hop penalty to reduce repeated assignments. This prioritizes nodes that maximize coverage efficiency while balancing local exploitation and global exploration. A supervisory planner loop continuously assigns goals to idle robots, first checking for completion and prioritizing stray edges before applying scoring-based selection. To maintain stability within Open-RMF, goal dispatching is rate-limited and recent assignments are tracked to prevent repetition. Goals that fail repeatedly are treated as unreachable and bypassed. Additional implementation-level details and execution logic for these algorithms are provided in Appendix B.

Evaluation

Simulations and Results

The proposed algorithm, Edge-Coverage, was evaluated across maps of 30, 50, and 70 nodes with fleets of 2–4 robots, and compared against modified random and frontier-based baselines. All methods were adapted to ensure full edge coverage, and performance was

measured using completion time and coefficient of variation (CoV) over five trials per map (15 maps total). A more detailed per-map analysis of CoV behaviour and its relationship with graph structure is provided in Appendix C.

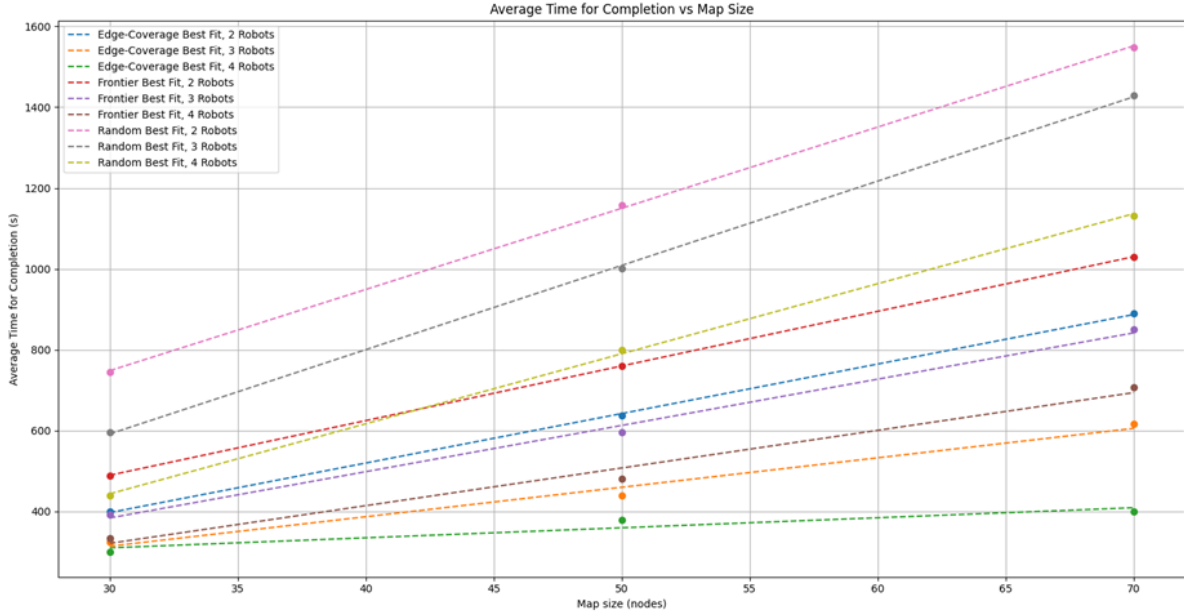


Figure 3: Graph of Average Completion Time vs Map size

As shown in Figure 3, Edge-Coverage consistently achieves the lowest completion times across all configurations. On 70-node maps with two robots, it reduces completion time by approximately 41% compared to random and 15% compared to frontier methods. With four robots, improvements increase to over 65% and 50% respectively. While all methods scale approximately linearly with map size, Edge-Coverage exhibits the smallest rate of increase, indicating superior scalability. Performance gains are most significant when increasing from two to three robots, with diminishing returns thereafter due to spatial constraints in smaller maps. This suggests that Edge-Coverage effectively minimizes redundant traversal and enables implicit coordination by distributing exploration across the graph.

Figure 4 shows that frontier-based exploration achieves the lowest CoV, followed by Edge-Coverage, while random exploration is the least consistent. Edge-Coverage remains stable in smaller configurations but exhibits higher variance in mid-sized maps, where multiple valid task assignments increase sensitivity to early decisions. In contrast, larger maps demonstrate improved stability. Increased connectivity reduces effective distances between nodes, constraining task allocation choices and producing more consistent outcomes. A key observation is that this variability does not originate from the algorithm itself, which is deterministic, but from the nondeterministic task allocation in Open-RMF. Differences in assignment arise from asynchronous updates, pose estimation noise, and internal tie-breaking, leading to divergent robot trajectories across runs. As a result, Edge-Coverage inherits system-level variability while maintaining strong efficiency gains. Despite this, the algorithm achieves its primary objectives: reducing redundant traversal, maintaining low dispatch overhead, and remaining fully compatible with real-time multi-robot coordination.

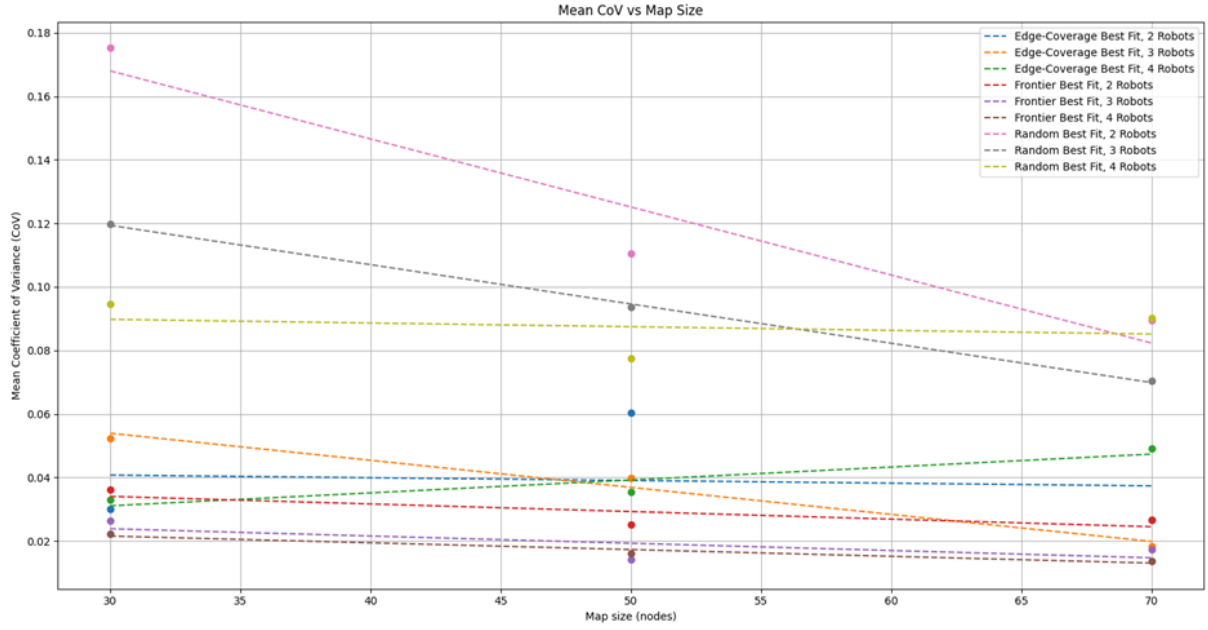


Figure 4: Graph of Average Completion Time vs Map size

Limitations

Several limitations affect this evaluation. The environments were simplified and did not include dynamic edge changes such as opening or closing paths. Experiments were also limited to maps of up to 70 nodes and fleets of four robots due to computational constraints, restricting large-scale scalability analysis. In addition, the number of maps and trials was constrained by time and simulation cost, limiting statistical breadth. Finally, evaluation metrics focused primarily on completion time and CoV, without considering factors such as load balancing, revisit frequency, or energy constraints. These limitations suggest that further work is needed to evaluate performance in larger, more dynamic, and resource-constrained settings.

Conclusion

This project developed a graph-aware goal-selection framework for multi-robot edge coverage within Open-RMF, improving coordination efficiency and reducing redundant traversal while remaining compatible with real-time middleware constraints. Experimental results demonstrate consistent improvements in completion time and scalability across different map sizes and robot configurations. Future work will focus on extending the framework to more realistic scenarios, including dynamic environments with lane closures, larger robot fleets, and additional evaluation metrics. Integration with real-time perception systems, such as YOLO-based object detection, could further enable task-driven exploration where robots identify and respond to targets during operation. Together, these improvements would advance the framework toward more robust, scalable, and practical multi-robot systems for applications such as inspection and search-and-rescue.

References

- [1] open-rmf, “GitHub - open-rmf/rmf_traffic: Traffic management libraries for RMF,” *GitHub*, 2025. https://github.com/open-rmf/rmf_traffic [Accessed: Nov. 16, 2025].
- [2] open-rmf, “GitHub - open-rmf/rmf_task: RMF library for managing task allocations,” *GitHub*, 2025. https://github.com/open-rmf/rmf_task [Accessed: Nov. 16, 2025].
- [3] J. Oñate-López, L. Navarro, C. G. Quintero M., and M. Pardo, “Intelligent exploration approaches based on utility functions optimization for multi- agent environment applications,” *Applied Sciences*, vol. 11, no. 5, 2021, issn: 2076-3417. doi: 10.3390/app11052408. [Online]. Available: <https://www.mdpi.com/2076-3417/11/5/2408>.
- [4] E. Tolstaya, J. Paulos, V. Kumar, and A. Ribeiro, Multi-robot coverage and exploration using spatial graph neural networks, 2021. arXiv: 2011.01119 [cs.RO]. [Online]. Available: <https://arxiv.org/abs/2011.01119>.
- [5] Kapoutsis, A.C.; Chatzichristofis, S.A.; Kosmatopoulos, E.B. DARP: Divide areas algorithm for optimal multi-robot coverage path planning. *J. Intell. Robot. Syst.* 2017, 86, 663–680.
- [6] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring network structure, dynamics, and function using networkx,” in *Proceedings of the 7th Python in Science Conference*, G. Varoquaux, T. Vaught, and J. Millman, Eds., Pasadena, CA USA, 2008, pp. 11–15.
- [7] M. Drobyshevskiy and D. Turdakov, “Random graph modeling: A survey of the concepts,” *ACM Computing Surveys*, vol. 52, no. 6, pp. 1–36, Dec. 2019, issn: 1557-7341. doi: 10.1145/3369782. [Online]. Available: <http://dx.doi.org/10.1145/3369782>.
- [8] Allender, E., Mix Barrington, D.A., Chakraborty, T. *et al.* Planar and Grid Graph Reachability Problems. *Theory Comput Syst* 45, 675–723 (2009). <https://doi.org/10.1007/s00224-009-9172-z>
- [9] Lobster Graph,” *Wolfram.com*, 2025. <https://mathworld.wolfram.com/LobsterGraph.html> [Accessed: Nov. 16, 2025].

Appendices

Appendix A: Graph Generation Details

This appendix provides additional implementation details and design considerations for the graph generation process used in this study, supporting Figures A1 and A2. While the main report outlines the overall approach, this section elaborates on the structural design choices, parameter tuning, and Open-RMF-specific configurations that enabled realistic and functional simulation environments.

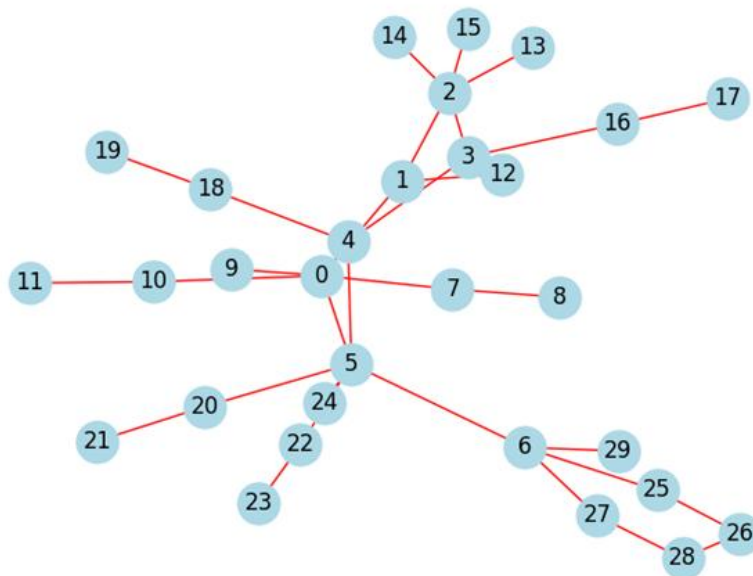


Figure A1: A modified lobster graph generated through NetworkX

The base graph structure used throughout this project is the lobster graph, a subclass of tree graphs in which all nodes lie within distance two of a central backbone or spine. This structure was chosen because it captures the hierarchical character of indoor environments such as corridors with branching rooms while preserving guaranteed planarity, making it suitable for both simulation and visualization.

Graph generation was implemented programmatically using NetworkX with custom routines controlling three key parameters. Spine length determines the primary traversal path and sets the overall scale of the environment. Branch count governs how many secondary paths extend from the spine, shaping layout density. Branch depth controls how far these paths extend, influencing exploration complexity. Together these parameters allow systematic variation in graph size and structure across experiments.

Although lobster graphs are inherently planar, their two dimensional embedding can still introduce ambiguity when nodes are positioned too closely. This becomes especially problematic in Open-RMF where robot localization depends on proximity to predefined waypoints. If nodes overlap or are insufficiently spaced, a robot may incorrectly associate its position with the wrong node, leading to navigation errors. To mitigate this, spatial layout constraints were applied during graph embedding, including minimum spacing thresholds and controlled positional offsets that ensure clear separation between adjacent nodes. These measures improve both simulation reliability and interpretability.

A further limitation of pure tree structures is the absence of cycles, which reduces realism compared to real indoor environments. This was addressed by introducing loops probabilistically by adding edges between randomly selected node pairs that were not already connected. The probability of loop formation was tuned to balance structural complexity with navigability, producing hybrid graphs that retain the hierarchical clarity of trees while incorporating cyclic paths commonly found in buildings.

To support fair performance comparison across different maps, normalization was performed using average edge weight. Rather than scaling graphs based only on physical dimensions or node count, this approach ensures consistent traversal cost across environments. Because edge weights correspond to travel distance or time, normalizing by their average isolates the effect of topology on algorithm performance, allowing differences in results to reflect structural properties rather than scale.

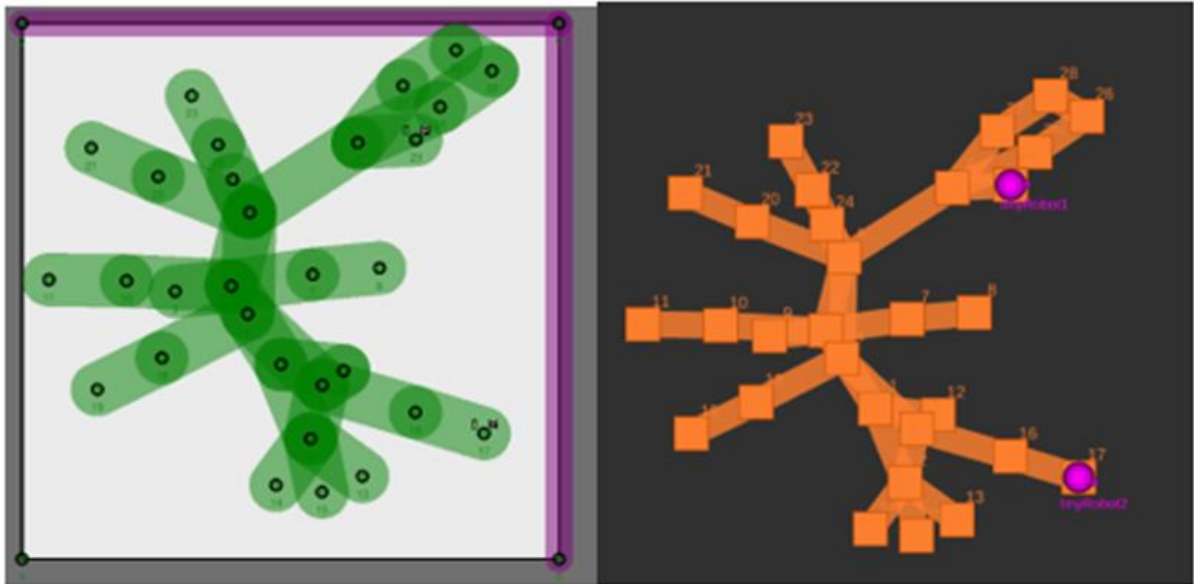


Figure A2: Randomly Generated Graph converted into building map for Open-RMF

Once the graph structure was finalized, it was converted into an Open-RMF compatible building map format. This process involved assigning semantic and operational properties to

nodes and edges so that the graph is not only structurally valid but also executable within the RMF ecosystem.

Each node in the graph was given a functional role. Leaf nodes were designated as parking and charging stations. The number of these nodes was matched to the number of robots in the simulation. These same nodes were also used as initial spawn locations, reflecting more realistic deployment scenarios.

Choosing leaf nodes for parking is important because placing robots at intermediate nodes can block multiple edges and disrupt navigation. By using leaf nodes, which have degree one, interference with ongoing exploration is minimized.

Each edge was configured according to Open-RMF constraints. Although RMF supports different edge types such as unidirectional lanes and mutex controlled regions, this study simplifies the configuration. All edges were set to bidirectional so robots could traverse freely without deadlocks. Human only and restricted edges were excluded to keep traversal assumptions consistent. Mutex groups were not included because their effects fall outside the scope of algorithm evaluation.

These simplifications ensured that any performance differences observed were driven by the goal selection algorithm rather than environmental constraints.

In addition to topology and connectivity, the graph was encoded into a `building.yaml` file, which serves as the primary map representation in Open-RMF. This file stores node coordinates, edge connections, and semantic annotations required for navigation and task planning. The conversion process was fully automated so that multiple test environments could be generated quickly and with consistent formatting.

Appendix B: Algorithm Pseudocode and Implementation Details

This appendix provides additional clarification and implementation level details for the pseudocode presented in Figures B1 and B2, covering the goal selection algorithm and the planner loop. While the main report focuses on conceptual design and performance impact, this section explains how the logic is structured in practice, how decisions are computed at runtime, and how the system integrates with Open-RMF and ROS 2 based execution constraints.

Figure B1 describes the goal selection algorithm, which determines the next optimal node for each robot using a combination of graph structure, coverage state, and traversal cost. Unlike simple frontier based approaches, this method explicitly incorporates coverage history, structural awareness, and multi objective scoring to improve coordination efficiency across multiple robots. At runtime, the algorithm evaluates every accessible node in the graph as a candidate and assigns each one a numerical score. The node with the highest score is selected as the next goal, ensuring that selection is globally informed rather than locally greedy.

The scoring function is composed of four main components. Exploration gain measures the number of unvisited edges connected to a node. Traversal cost estimates the shortest path distance from the robot's current position. Fragment bonus prioritises partially explored regions to encourage completion of local subgraphs. Hop penalty discourages repeated oscillation between nearby nodes or recently visited goals. These terms are combined into a single evaluation function that balances exploration efficiency with travel minimisation.

A key design choice is that the algorithm does not compute globally optimal coverage paths. Instead, it approximates global efficiency through repeated local evaluations, which makes it suitable for real time execution inside Open-RMF dispatch cycles. Special cases are handled explicitly. Nodes with no remaining unvisited edges are automatically deprioritised. Recently visited nodes are temporarily penalised to reduce oscillatory behaviour. When multiple nodes obtain similar scores, tie breaking is resolved using shortest path cost or deterministic ordering to ensure reproducibility. This keeps the method lightweight while still reflecting global structure through incremental decision making.

Algorithm 1 Edge-Coverage Goal Selection Algorithm

Require: Graph $G = (V, E)$, Robot positions $R = \{r_1, \dots, r_m\} \subseteq V$, Visited nodes $V_{\text{visited}} \subseteq V$, Visited edges $E_{\text{visited}} \subseteq E$, Excluded goals $V_{\text{excluded}} \subseteq V$

Ensure: Selected goal $g^* \in V$

```
1:  $G_c \leftarrow V \setminus V_{\text{excluded}}$  ▷ Candidate nodes
2:  $g^* \leftarrow \emptyset, \text{best\_score} \leftarrow -\infty, O \leftarrow \emptyset$ 
3: for each  $g \in G_c$  do
4:    $\text{gain}(g) \leftarrow |\{e = (g, v) \in E \mid e \notin E_{\text{visited}}\}|$ 
5:   if  $\text{gain}(g) = 0$  then continue
6:   end if
7:   for each  $r \in R$  do
8:      $P_{r,g} \leftarrow$  shortest path from  $r$  to  $g$ 
9:      $\text{cost}(P_{r,g}) \leftarrow \sum_{e \in P_{r,g}} w(e)$ 
10:     $\text{new\_edges}(P_{r,g}) \leftarrow |\{e \in P_{r,g} \mid e \notin E_{\text{visited}}\}|$ 
11:   end for
12:    $f(g) \leftarrow \begin{cases} 0.5 & g \in V_{\text{visited}} \\ 1.0 & \text{otherwise} \end{cases}$ 
13:    $h(g) \leftarrow \sum_{v \in V_{\text{excluded}}} \text{penalty\_hop}(d(g, v))$ 
14:    $\text{score}(g) \leftarrow \frac{\text{gain}(g) \cdot \text{new\_edges}(P_{r,g}) \cdot f(g)}{\text{cost}(P_{r,g})} - h(g)$ 
15:   Add  $o_g \leftarrow (g, \text{score}(g))$  to  $O$ 
16:   if  $\text{score}(g) > \text{best\_score}$  then
17:      $g^* \leftarrow g, \text{best\_score} \leftarrow \text{score}(g)$ 
18:   end if
19: end for ▷ Refinement: Leaf detection
20: if  $g^*$  has degree 2 then
21:   Follow unbranched neighbors until leaf  $\ell$  (degree 1)
22:   if  $\ell \notin V_{\text{visited}}$  then  $g^* \leftarrow \ell$ 
23:   end if
24: end if ▷ Refinement: Single-exit cycle detection
25: if  $g^*$  belongs to cycle  $C$  with one exit node then
26:   Select farthest unvisited node along  $C$  from the exit
27: end if
28: return  $g^*$ 
29: function PENALTY_HOP( $d$ )
30:   if  $d = 1$  then return 0.5
31:   else if  $d = 2$  then return 0.2
32:   else return 0
33:   end if
34: end function
```

Figure B1: Goal Selection Algorithm

Figure B2 describes the planner loop, which acts as the supervisory control mechanism that continuously monitors robot states and triggers goal assignments. It runs at a fixed update interval to avoid excessive task dispatching, since high frequency injection can degrade performance in Open-RMF.

At each iteration, the loop first checks whether all edges in the graph have been visited. If this condition is satisfied, the mission is terminated and no new goals are generated. Next, robot states are evaluated to determine whether each robot is actively executing a task, idle and available, or stuck due to lack of progress or timeout conditions. Only idle robots are eligible for new assignments.

If unvisited stray edges remain, meaning edges that are isolated due to fragmented traversal, they are prioritised before general goal selection. This prevents partial coverage from persisting until the end of execution. If no urgent edges exist, the planner invokes the goal selection algorithm from Figure B1 to compute the next optimal node for each available robot. Dispatching is then rate limited to maintain system stability. A short history of recent assignments is maintained and duplicate or overly frequent goal submissions are suppressed to avoid overloading the RMF task queue.

Failure handling is included to improve robustness. If a robot repeatedly fails to progress toward a target, the system assumes that region is temporarily unreachable. The affected edges are marked as visited or deprioritised, and the robot is reassigned to alternative regions. The planner also avoids reselecting the same goal in the short term.

Both components integrate directly into the Open-RMF dispatch pipeline. The planner loop communicates with RMF's task submission system, while the goal selection module operates independently of low level motion planning. This separation is intentional, with RMF handling execution and scheduling, and the proposed algorithm handling decision making. The system is also compatible with Navigation2, ensuring that generated goals can be executed using standard navigation pipelines.

From a computational standpoint, the algorithm remains suitable for real time operation. Node evaluation scales linearly with the number of candidate nodes, no global re-optimization is performed at each step, and planning is distributed across iterative updates. Complexity is further controlled through rate limiting and filtering of inactive robots, allowing the approach to scale to medium sized multi robot systems.

Algorithm 2 Planner Loop

Require: Graph $G = (V, E)$, Robot set $R = \{r_1, \dots, r_m\}$, Robot positions $pos(r)$, Visited nodes $V_{visited}$, Visited edges $E_{visited}$, Recently dispatched goals G_{recent}

Ensure: Update tasks and plan next move

- 1: **if** all edges visited, i.e., $E_{visited} = E$ **then**
- 2: Log task completion with elapsed time
- 3: **return**
- 4: **end if**
- 5: **for** each robot $r \in R$ **do**
- 6: $n \leftarrow pos(r)$ ▷ Current node of robot r
- 7: **if** n is undefined **then**
- 8: Log warning and **continue**
- 9: **end if**
- 10: **if** robot r is busy **then continue**
- 11: **end if**
- 12: $stray \leftarrow$ Find stray edge goal near n ▷ Refinement: Stray edge detection
- 13: **if** $stray \neq \emptyset$ **then**
- 14: Dispatch goal $stray$
- 15: Add $stray$ to G_{recent}
- 16: **continue**
- 17: **end if**
- 18: $C \leftarrow$ Candidate nodes in G not in $V_{visited}$ or G_{recent}
- 19: $goal \leftarrow \text{SelectBestGoal}(G, C, pos(R), E_{visited}, V_{visited})$
- 20: **if** $goal \neq \emptyset$ **then**
- 21: **if** $goal \notin G_{recent}$ **then**
- 22: Dispatch goal $goal$
- 23: Add $goal$ to G_{recent}
- 24: **else**
- 25: Mark edges around $goal$ as visited
- 26: **end if**
- 27: **end if**
- 28: **end for**
- 29: Schedule next planner loop after delay

Figure B2: Planner Loop Algorithm

Appendix C: Detailed Analysis of CoV Distribution Across Map Instances

Appendix C provides an extended interpretation of Figure C1, which shows the distribution of Coefficient of Variation across individual maps for different robot configurations. While the main report focuses on aggregated trends and overall algorithm comparison, this figure reveals finer grained behaviour that depends strongly on map structure, robot density, and the task allocation dynamics of Open-RMF.

Figure C1 presents a per map breakdown of CoV values across all tested configurations, allowing stability to be analysed at the level of individual graph instances rather than global averages. This makes it clear that Edge Coverage does not behave uniformly across environments. Instead, its stability emerges from the interaction between graph topology and multi robot task assignment density, rather than from any single controlling factor.

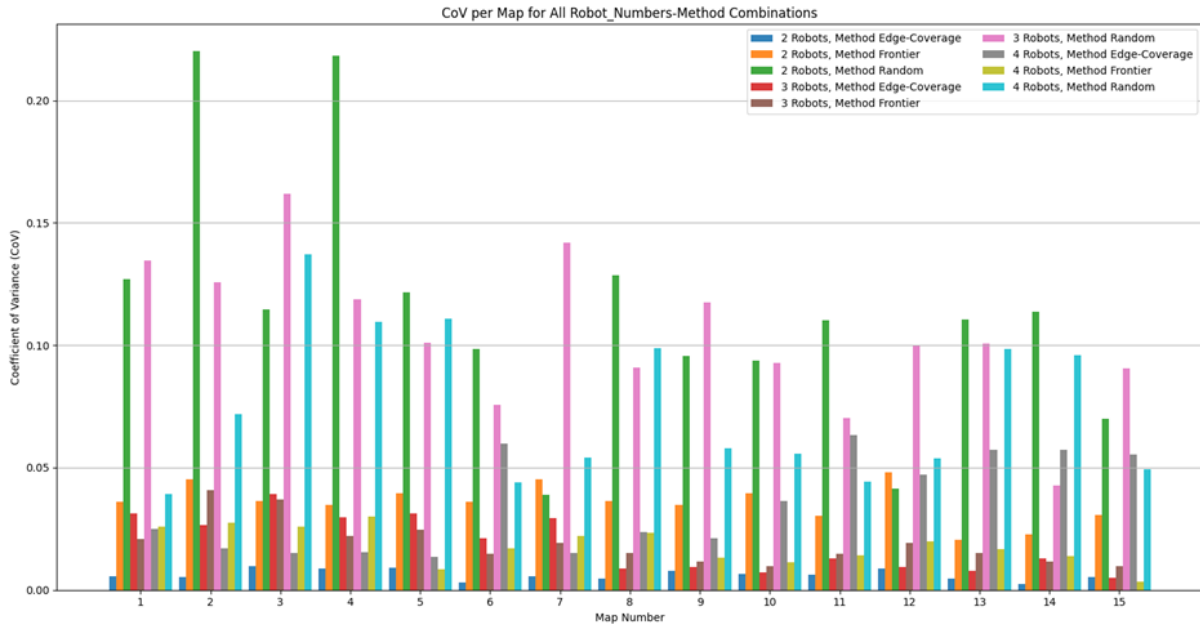


Figure C1: Distribution of CoV for different maps

For the two robot configurations, Edge Coverage consistently exhibits low CoV across all maps, indicating highly stable execution with minimal run to run variation. In some cases it even outperforms Frontier in higher robot density settings in terms of stability. This behaviour can be explained by the limited combinatorial space of task assignments within Open-RMF. With only two agents, there are few competing allocation permutations, which reduces sensitivity to small fluctuations in scheduling decisions, pose updates, and internal tie breaking. As a result, even though RMF retains nondeterministic elements, their impact on global trajectories is strongly constrained in low density scenarios and is quickly absorbed by the deterministic structure of the scoring function.

In the fifty node maps (maps 6 to 10), CoV becomes highly unstable for both three and four robot configurations. This instability is driven by a combination of high allocation entropy

and structural flexibility in the graph. The environment contains many simultaneously valid goal assignments, and its branching structure allows multiple equivalent exploration routes. In this regime, small differences in early task allocation within Open-RMF are repeatedly amplified over time. Even minor variations such as asynchronous communication delays, slight differences in robot pose updates, or internal tie breaking decisions can redirect robots onto different exploration paths. Because these early divergences occur when the system has maximum assignment freedom, they propagate through the remainder of the run and result in large variability in completion order and traversal history.

By contrast, the seventy node maps (maps 11 to 15) show a reduction in CoV for both three and four robot cases despite increased graph size. This stabilisation arises from changes in effective topology rather than reduced complexity. Additional nodes introduce more loops and alternative connections, which reduce effective graph diameter and shorten topological distances between robots and tasks. Under the RMF distance based heuristic, this leads to fewer strongly competing assignment options because most tasks become similarly close to at least one robot. The decision space becomes more constrained even though the environment is larger, reducing the sensitivity of allocation outcomes to small fluctuations in state.

At this scale, the modified lobster graphs begin to resemble Barabási Albert like structures, where a small number of nodes act as hubs with disproportionately high connectivity. These hubs compress effective travel distances and reduce the independence of robot trajectories by funnelling movement through common regions. This reduces allocation entropy because many tasks are no longer meaningfully distinct in terms of assignment cost, which in turn limits divergence across runs.

A key insight from Figure C1 is that the observed variability is not primarily a property of the Edge Coverage algorithm, which remains deterministic, but instead arises from the nondeterministic behaviour of the underlying Open-RMF task allocation layer. Across identical conditions, RMF may assign different robots to equivalent tasks, occasionally select non minimal distance allocations, or produce different assignment sequences under similar cost conditions. These variations are influenced by asynchronous communication timing, small fluctuations in pose estimation, and internal scheduling heuristics that do not guarantee consistent outcomes.

Because task allocation directly determines robot trajectories, even small differences in assignment propagate into large downstream differences in coverage order and timing. This explains why variance peaks in mid-sized graphs where allocation ambiguity is highest, and why larger graphs show improved stability due to reduced effective decision entropy.

Overall, Figure C1 demonstrates that Edge Coverage stability is highly context dependent. It performs consistently in low robot density settings, becomes unstable in mid sized graphs due to maximum allocation ambiguity, and regains stability in larger graphs where structural constraints reduce decision freedom. Most importantly, it shows that performance variability is governed by the interaction between graph topology, robot density, and Open-RMF scheduling behaviour rather than algorithm design alone.

Appendix D: Gantt Chart

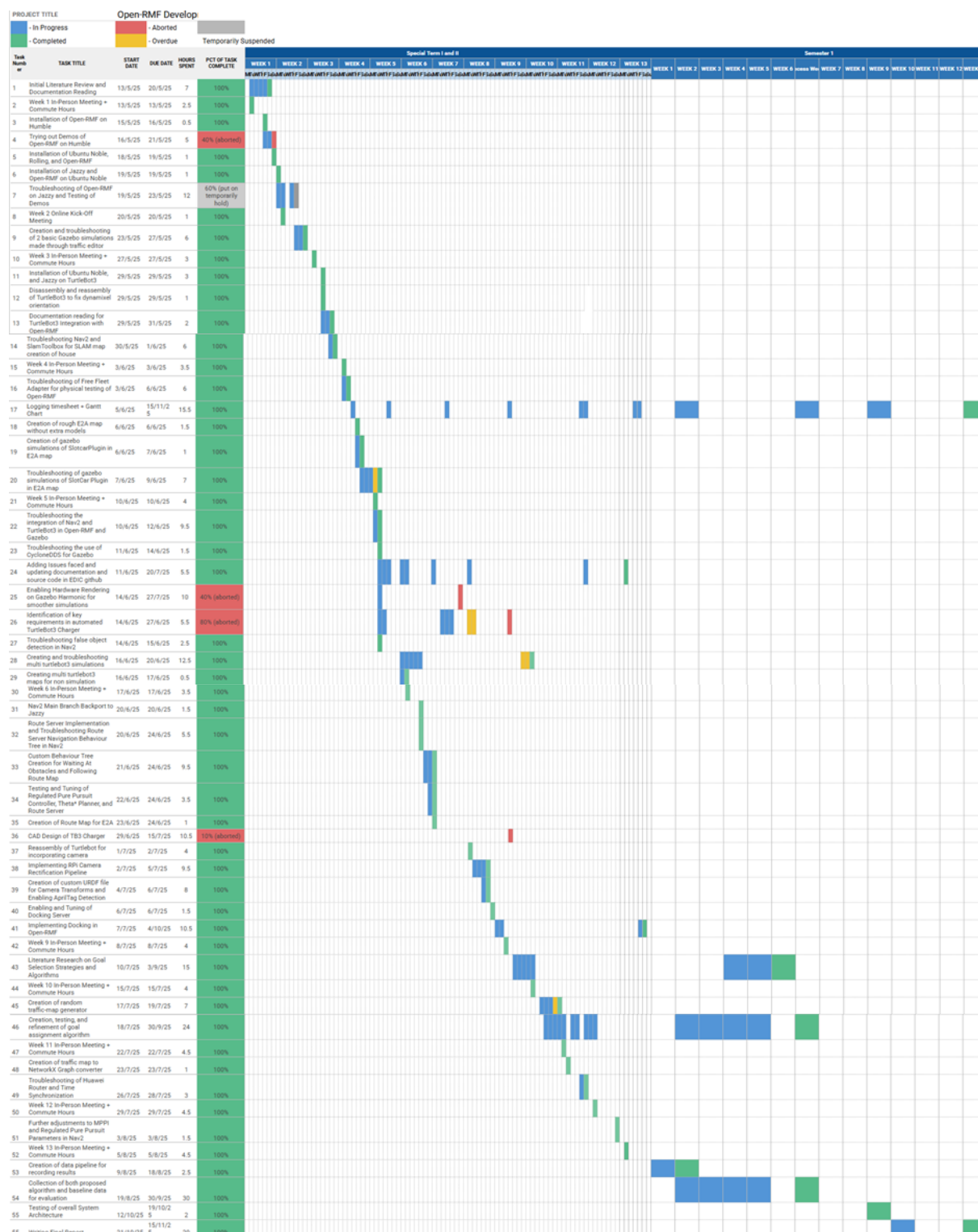


Figure D1: Gantt Chart